

Halocline

The theory in plain language, and how we build it

Problem Statement SIH26066 — OceanEmbed

Organisation Ministry of Earth Sciences · INCOIS, Ocean Valley

Category Software | Theme: Disaster Management

Domain North Indian Ocean, 5°N-30°N / 45°E-105°E

Team 6 members — ML, data, backend, frontend

Contents

PART 1 — THE WHOLE PROJECT IN ONE PAGE	3
PART 2 — OCEAN THEORY, ONLY THE PARTS WE USE	4
2.1 The ocean has three layers	4
2.2 The barrier layer, and why the Bay of Bengal is special	4
2.3 Temperature inversions — the trap	4
2.4 Depth of the 26 °C water (D26) and heat content (TCHP)	5
2.5 Why surface data can predict the deep ocean	5
2.6 Density needs both temperature and salinity	5
2.7 What we train against, and the honesty problem	6
2.8 The four things we measure	6
2.9 Two oceans, one model	6
PART 3 — MACHINE LEARNING, ONLY THE PARTS WE USE	7
3.1 What goes in and what comes out	7
3.2 Predict the difference, not the value	7
3.3 The depth decoder — our main technical idea	7
3.4 The loss function — three parts	8
3.5 Splitting the data — the mistake that kills projects like this	8
3.6 Baselines — do not skip these	8
3.7 Is the uncertainty honest?	9
PART 4 — THE JURY QUESTION, AND THE ANSWER	10
PART 5 — SYSTEM ARCHITECTURE	11
The single most important architectural decision	11
The three contracts — agree these on day one	11
PART 6 — FRONTEND	12
6.1 Design direction	12
6.2 Layout	13
6.3 The five screens	13
6.4 Stack	13
6.5 How the map layer actually works	14
PART 7 — BACKEND	15
PART 8 — ML PIPELINE	16
PART 9 — WHO DOES WHAT	17
PART 10 — TIMELINE	18
PART 11 — THE FIVE-MINUTE DEMO	19
PART 12 — THINGS THAT WILL GO WRONG	20

PART 1 — THE WHOLE PROJECT IN ONE PAGE

What we are building. Satellites can only see the skin of the ocean. We take what satellites see at the surface, and predict what the water looks like all the way down to 1000 m — temperature and salinity at every depth, every day, across the Bay of Bengal and Arabian Sea.

Why anyone cares. Cyclones do not feed on surface temperature. They feed on the *depth* of the warm water. A storm's winds stir the ocean like a spoon in a cup. If the warm layer is thin, the stirring pulls up cold water from below, the surface cools, and the storm starves. If the warm layer is deep, the stirring only brings up more warm water, and the storm explodes in strength.

Two spots in the Bay of Bengal can both read 30 °C from space and behave completely differently, because one has 30 m of warm water under it and the other has 100 m. **A satellite cannot tell those two spots apart. Our model can.**

Why it is hard. The only direct measurements of the deep ocean come from Argo floats — drifting robots that dive and surface. There is roughly one float per 3° square, reporting once every 10 days. Almost the entire basin is unmeasured almost all of the time.

Why it is possible. Warm water expands. Where the warm layer is thick, the sea surface physically bulges upward by a few centimetres. Satellite altimeters measure that bulge to about 2 cm. So sea surface height is effectively a tape measure for how deep the warm water goes. That one fact is the scientific foundation of the entire project.

Our name. The *halocline* is the layer where saltiness changes sharply with depth. In the Bay of Bengal it is created by river water and it is the reason this basin behaves differently from everywhere else on Earth. It is the thing our model treats specially and nobody else does.

PART 2 — OCEAN THEORY, ONLY THE PARTS WE USE

Nine ideas. Each one is here because it turns into a line of code or a slide. Nothing else.

2.1 The ocean has three layers

Picture a cup of tea you just stirred.

- **Mixed layer** — the top. Wind and waves stir it, so it is the same temperature from top to bottom. Usually 10–120 m thick.
- **Thermocline** — below it, temperature falls off a cliff. Roughly 50–200 m here. **Almost everything interesting happens in this layer.**
- **Deep ocean** — below ~500 m. Cold, slow, barely connected to the surface on a day-to-day basis.

Why we care: our model will be excellent between 20 m and 200 m, and useless at 700 m and 1000 m. That is not a bug we can fix — the information simply is not in the surface data. We must say this openly and show it as a number.

In the code: we report a *skill score* at every depth separately, rather than one average error for the whole water column. An average would hide the good and the bad in each other.

2.2 The barrier layer, and why the Bay of Bengal is special

The Ganges, Brahmaputra and Irrawaddy dump enormous amounts of fresh water into the Bay of Bengal. Fresh water is lighter than salty water, so it floats on top, like oil on water.

That floating fresh lid is hard to mix through. Wind can stir the top few metres, but it cannot punch down into the warm water below. The layer between “as far as stirring reaches” and “where the temperature actually drops” is the **barrier layer**. It acts as a lid that protects the warm water underneath from being disturbed.

Three definitions we actually compute:

- **MLD** (mixed layer depth) — how deep the *density* stays roughly constant. Stirring reaches this far.
- **ILD** (isothermal layer depth) — how deep the *temperature* stays roughly constant.
- **BLT** (barrier layer thickness) = ILD – MLD. When this is large, there is a protected warm layer, and a cyclone over it can intensify without limit.

The number that makes the case: in seawater, 1 unit of saltiness is worth about 3 °C in terms of how heavy the water is. So in the Bay of Bengal, **saltiness controls the layering, not temperature**. Almost every global model underweights this. We do not.

2.3 Temperature inversions — the trap

In winter in the northern Bay of Bengal, the surface loses heat to the air and gets cold. Normally cold surface water would sink and mix. But the fresh lid is so light that it stays on top even when it is cold.

The result: water gets *warmer* as you go deeper, for the first few tens of metres. Inversions of 0.5–2 °C are completely normal there.

Why this matters enormously: the intuitive thing to do in the model is to force temperature to always decrease with depth. **We must not do that.** It would delete the exact phenomenon we claim to specialise in. We force profiles to be *smooth*, never to be *always decreasing*.

2.4 Depth of the 26 °C water (D26) and heat content (TCHP)

- **D26** — how deep you have to go before the water drops below 26 °C. Cyclones cannot run on water colder than that.
- **TCHP** — how much heat is stored above that depth. Think of it as how much fuel is in the tank.

Rough operational numbers: about 50 kJ/cm² supports a cyclone strengthening; 90 and above is associated with rapid intensification.

In the code: we compute D26 and TCHP from our predicted profiles and show them as maps. These are the quantities INCOIS and IMD genuinely use, so we are judged on them, not just on raw temperature error.

2.5 Why surface data can predict the deep ocean

Four inputs, four different jobs:

- **Sea surface height (SLA)** — the most important one by far. Warm water expands, so a thick warm layer makes the sea surface bulge. Also, swirling eddies 100–300 km wide push the thermocline up and down by tens of metres, and that shows up directly in surface height. This is how the model “sees” downward.
- **Sea surface temperature (SST)** — tells us about the top of the profile.
- **Sea surface salinity (SSS)** — tells us about the fresh lid, so it tells us about the barrier layer. This is the Bay of Bengal-specific input.
- **Winds and currents** — winds set how hard the ocean is being stirred and push the thermocline up or down; currents carry water around.

This is not a new idea — the navy has been generating synthetic profiles from surface height for decades, and there is a 2004 paper doing it with neural networks in the Indian Ocean. **Our contribution is not “ML can do this.”** It is: continuous depth, temperature and salinity together, saltiness-aware physics, and honest uncertainty, in the one basin where saltiness is the deciding factor.

2.6 Density needs both temperature and salinity

How heavy seawater is depends on temperature *and* saltiness. You cannot compute it from temperature alone.

Consequence: if we want to check that our predicted profiles are physically sensible — heavy water below light water, never the reverse — we are forced to predict salinity too, not just temperature. This is why our model has two outputs instead of one. It is also why the project is called Halocline.

In the code: the `gsw` Python package for all density calculations. It is the community standard, and using it signals to an ocean scientist that we know what we are doing.

2.7 What we train against, and the honesty problem

We cannot train on Argo floats — there are far too few. So we train on **GLORYS**, a physics model that has been nudged into agreement with every observation ever collected. It gives us a complete ocean, every day, everywhere.

The problem: GLORYS has already absorbed the Argo float data. So when we later “validate against independent Argo floats”, the floats are not independent — GLORYS already knew about them, and we learned from GLORYS. Holding floats out of *our* training does not fix this. It is baked in.

This is the hardest question the jury will ask. The answer we give is in Part 4. Do not improvise it.

2.8 The four things we measure

For every depth, against real Argo profiles:

- **RMSE** — typical size of our error, in °C.
- **Bias** — are we systematically too warm or too cold.
- **Correlation** — do we get the ups and downs right.
- **Skill score** — how much better we are than simply guessing the long-term average for that place and time of year. This is the one that actually matters. A skill score of 0 means we added nothing.

Plus the derived ones that decide cyclone behaviour: MLD, BLT, D26, TCHP.

2.9 Two oceans, one model

The Arabian Sea is the opposite of the Bay of Bengal — more evaporation, saltier surface, no fresh lid, deeper mixing. One model has to handle both.

In the code: every metric is reported separately for the Bay of Bengal and for the Arabian Sea. If we only report a basin average, we will be asked why, and there is no good answer.

PART 3 — MACHINE LEARNING, ONLY THE PARTS WE USE

3.1 What goes in and what comes out

In: a small square patch of the ocean surface — 32×32 grid cells, which is $8^\circ \times 8^\circ$, about 900 km across — with 7 channels stacked (SST, SSS, SLA, two current components, two wind components), plus the day of the year.

Why a patch instead of a single point: how deep the thermocline sits at one spot depends on the eddies swirling *around* that spot. A single-point model throws away that spatial information, which is most of the signal.

Out: temperature and salinity at any depth you ask for, each with an uncertainty.

3.2 Predict the difference, not the value

Do not ask the network to learn that the tropics are warm. It should not waste capacity on things we already know.

Instead, we subtract the long-term average for that place and that day of the year — the **climatology** — from both the inputs and the target. The network only has to learn the *deviation* from normal.

Three benefits:

- Much easier learning problem.
- “Beating the climatology baseline” becomes the literal training objective instead of something we claim afterwards.
- At 1000 m where there is no signal, the network outputs approximately zero deviation, which means it falls back gracefully to the climatological average instead of producing nonsense.

3.3 The depth decoder — our main technical idea

The obvious approach is 15 separate outputs, one per required depth. We do not do that.

Instead the network learns **one continuous curve**. You hand it a depth — any depth, including ones it never saw — and it returns the temperature there.

Three reasons this is better, all of which go in the pitch:

1. **Smooth, physical profiles.** Fifteen independent outputs can disagree with each other and produce a jagged, physically impossible profile. One curve cannot.
2. **We can enforce smoothness directly.** Because the curve is continuous, we can differentiate it and penalise sharp kinks. Fifteen separate outputs give you no derivative to work with.
3. **Any depth, free.** The problem statement asks for 15 levels. We can output 500. Good demo moment.

One catch: neural networks are lazy and prefer to draw gentle curves. Fed raw depth, the network produces an over-smoothed profile with the thermocline smeared out — exactly the feature we need. Two standard fixes, both of which we use:

- **Fourier features** — feed depth in as a set of sine and cosine waves at different frequencies instead of a single number. This lets the network draw sharp bends.
- **Stretched depth axis** — feed $\log(1 + \text{depth}/20)$ instead of raw metres, so the network spends its capacity near the surface where all the variability is, instead of spreading it evenly down to 1000 m.

3.4 The loss function — three parts

Part one: CRPS. Our model outputs a mean and an uncertainty. CRPS is a score that rewards being accurate *and* being honest about how confident you are. Claiming ± 0.1 °C and being wrong by 2 °C is punished harder than admitting ± 2 °C. For a bell-curve prediction it has an exact formula, so it costs one line of code and no extra compute.

Part two: smoothness. Penalise sharp kinks in the profile. Note again: smooth, not always-decreasing.

Part three: physical sanity. Penalise cases where the model predicts light water sitting below heavy water, which cannot happen in the real ocean. This needs salinity, which is why we predict it.

One more detail: temperature varies by several degrees at 30 m and by hundredths of a degree at 1000 m. If we do not rescale, the loss is entirely dominated by the shallow levels. So we divide each depth's error by how much that depth naturally varies. The model then trains directly on skill score.

3.5 Splitting the data — the mistake that kills projects like this

Today's ocean looks almost exactly like yesterday's ocean. If we shuffle all the days randomly into train and test sets, our test set is effectively sitting inside our training set, and we will report a beautiful error number that means nothing.

We split by time — train on years up to 2018, validate on 2019, test on 2020–21. **And by region** — hold out an entire geographic box the model never sees.

Never randomly.

3.6 Baselines — do not skip these

Our headline number is meaningless on its own. We report it beside:

1. **Climatology** — just guess the seasonal average. The bar we must clear.
2. **Linear regression** on SST + SLA + SSS, one per depth.
3. **Gradient boosting** on the same inputs.

If the deep learning model does not beat #3, we do not have a result. Showing the baselines is a credibility signal — it tells the jury we tested ourselves honestly. Hiding them tells them we did not.

3.7 Is the uncertainty honest?

Three checks, all quick:

- **Coverage** — our stated 95% range should contain about 95% of real observations. If it contains 60%, we are overconfident.
- **Spread vs error** — where we say we are unsure, we should actually be more wrong.
- **PIT histogram** — a flat bar chart means well-calibrated. A U-shape means overconfident.

The best single slide in the whole deck would be an uncertainty map where the river plumes are visible, because satellite salinity is unreliable there and the model correctly says so on its own.

PART 4 — THE JURY QUESTION, AND THE ANSWER

Question: “Aren’t you just copying GLORYS?”

The honest answer, in this order:

“Partly, yes, by construction — and here is the table that measures exactly how much.”

Then show one figure. Three rows, compared against the same set of real Argo profiles:

RMSE vs Argo at 100 m	
Climatology (guess the average)	worst
Halocline (ours, satellites only)	close to GLORYS
GLORYS (reanalysis, weeks of latency, uses the floats)	best

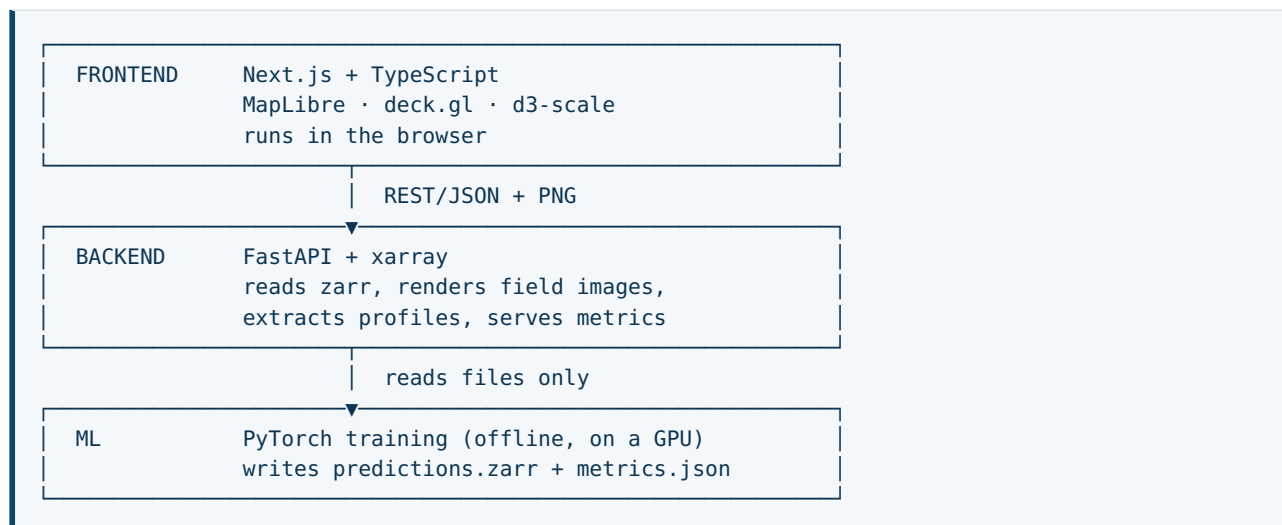
“GLORYS needs weeks of processing time and the in-situ float data to produce this field. We produce a comparable field from *today’s* satellite data alone, with calibrated uncertainty. Here is how much skill we give up for that, and here is what we gain.”

Then the finishing move: **run the model on satellite data from a period after GLORYS coverage ends, and check it against Argo floats from that period.** There is no GLORYS there at all. That comparison is genuinely clean, and it proves the operational claim directly.

Never say “we beat GLORYS.” It is not true, the jury will know it is not true, and it makes everything else we say suspect.

PART 5 — SYSTEM ARCHITECTURE

Three separate pieces, three separate teams, three clean contracts between them.



The single most important architectural decision

The website never runs the neural network.

Before the demo, the ML team runs the model once over the whole test period and saves every prediction into a file. The backend just reads that file. This means:

- The map responds instantly instead of in three seconds.
- Nothing can crash during the demo.
- It works with the wifi unplugged.
- The frontend team can build against real data from day 3 without needing a GPU.

We keep **one** live-inference endpoint, used for exactly one moment in the demo: the near-real-time panel, where we pull today's satellite data and run the model in front of the jury. That is the moment that proves it is real. Everything else is precomputed.

The three contracts — agree these on day one

ML → Backend: `predictions.zarr` with dimensions (time, depth, lat, lon) and variables `t_mean`, `t_std`, `s_mean`, `s_std`; plus `metrics.json`; plus `model.pt`.

Backend → Frontend: a fixed set of endpoints, documented in OpenAPI, plus a `fixtures/` folder of hand-written example responses so the frontend can be built before the backend works.

Everyone → Everyone: a `make_fake_data.py` that generates all of the above filled with random numbers in the correct shapes, committed in the first two hours.

This is the trick that lets six people work at once instead of five people waiting for one.

PART 6 — FRONTEND

Streamlit will not win this. Every second team has a Streamlit dashboard and they all look identical. We build a real web application.

6.1 Design direction

The concept: the page is a water column.

Almost every dashboard is organised left to right. Ours is organised top to bottom, because depth is the thing the project is about. A vertical depth scale runs down the left side of the screen — it is both the axis and the control. Drag it, and the map changes depth. The profile chart on the right uses the *same* vertical scale with the *same* tick positions, so your eye can read straight across the screen at a constant depth. Nobody else will have thought of this, and it takes ten minutes to explain why it is right.

Palette. Not another dark dashboard with a neon accent. We go the other way: a cool paper-white ground, like a printed bathymetric chart. Scientific data is printed on white, and the temperature field — which is vivid orange through blue — is far more striking against paper than against black.

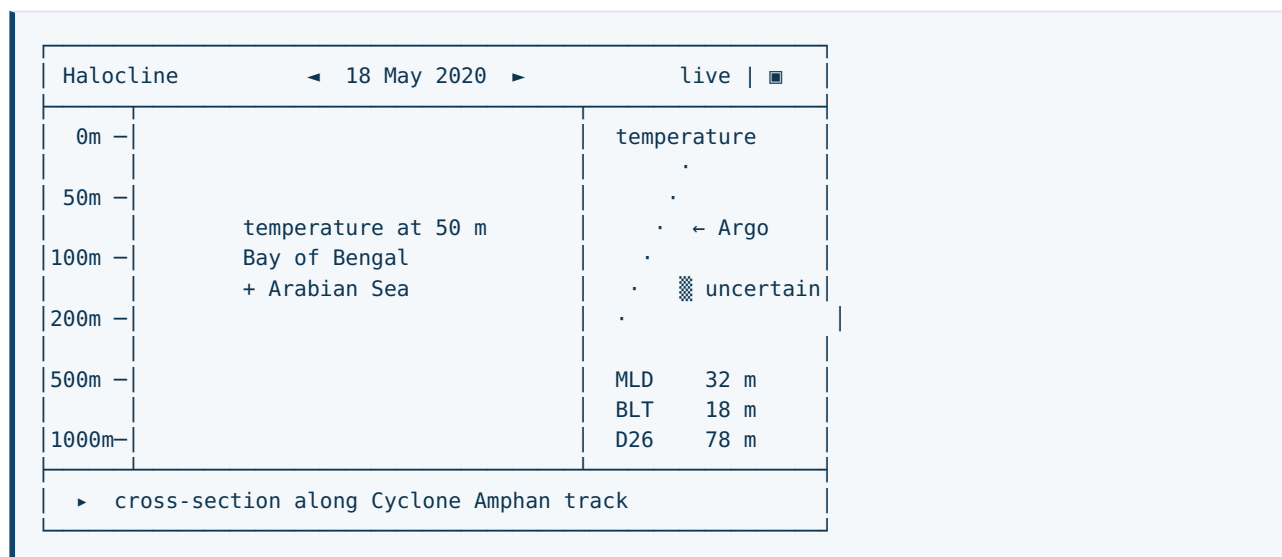
- Ground `#F2F4F5` · Ink `#10202B` · Rule `#C7D0D6` · Accent `#0B6E8F` · Warning `#B5651D`

Data colours: use cmocean. `thermal` for temperature, `haline` for salinity. These are the actual colormaps the ocean science community standardised on, and they are perceptually uniform, meaning equal colour steps represent equal temperature steps. An INCOIS judge will recognise them instantly. Using matplotlib's default `jet` or `viridis` would mark us as outsiders. This is a small detail that buys a lot of credibility.

Type. IBM Plex Sans for interface, IBM Plex Mono for all numbers. Plex was designed for technical interfaces, it is free, and it is not the Inter-everywhere default. Set numbers in tabular figures so values do not jitter as the depth slider moves.

Motion. One orchestrated moment only: the reveal of the 3D cross-section. Everything else responds to a click or a drag. No fade-up-on-scroll on every card — that is the generic default and it reads as unconsidered.

6.2 Layout



Click anywhere on the map and the right panel fills with that location's full profile, our uncertainty band, and — if an Argo float happened to be nearby that week — the real measurement overlaid as dots. **Seeing our curve pass through real float measurements is the most persuasive thing in the entire demo.**

6.3 The five screens

1. **Explorer** — the layout above. The default view.
2. **Cyclone** — pick Amphan (May 2020). The storm track animates across the map; below it, a vertical slice of the ocean along the track opens in 3D, with the 26 °C boundary drawn as a surface. You can see the deep warm pool the storm passed over. Beside it, a plot of SST along the same track that stays almost flat — **this is the picture that proves the whole premise.**
3. **Validation** — the three-way comparison table, error by depth, calibration plots, Bay of Bengal vs Arabian Sea broken out. Do not hide this behind a menu; putting our own error numbers front and centre is a statement.
4. **Live** — today's satellite data in, subsurface field out, in front of the jury, with the elapsed time shown. The only screen that calls the model.
5. **Method** — one scrollable page explaining the approach, for judges who read rather than watch.

6.4 Stack

- **Next.js 14 + TypeScript + Tailwind** — app shell
- **MapLibre GL JS** — the map. Free, no account or token needed, unlike Mapbox.
- **deck.gl** — the 3D cross-section
- **d3-scale + hand-built SVG** — for the profile chart. Not a charting library, because the chart has to share its exact vertical scale with the depth rail, and that is much easier to control by hand than to fight a library over.
- **Zustand** — shared state (selected date, depth, point)

6.5 How the map layer actually works

We do not build a tile server. The region is fixed, so the backend renders the whole domain as a single PNG (240 × 100 pixels, about 10 KB) and the frontend drops it onto the map as an image pinned to fixed corner coordinates. Changing depth means requesting a different PNG. It is fast, it is about thirty lines of code, and it will not fail on stage.

PART 7 — BACKEND

FastAPI. Its job is to turn files into JSON and PNGs, and nothing else.

Endpoint	Returns
GET /api/field	PNG of temperature or salinity at a given date and depth
GET /api/profile	full profile at a lat/lon: mean, uncertainty, derived MLD/BLT/D26/TCHP
GET /api/argo/nearby	real float profiles near a point and date, for overlay
GET /api/derived	PNG of D26 or TCHP for a date
GET /api/metrics	the validation tables and calibration data
GET /api/cyclone/amphan	track, and the ocean cross-section along it
POST /api/nrt/run	the only endpoint that runs the model

Things worth doing:

- Colour mapping with a cmocean lookup table, so the frontend never has to know about colours.
 - Cache rendered PNGs on disk. Pre-warm every date and depth used in the demo before we leave for the venue.
 - `docker-compose up` starts everything. Test it on a laptop with wifi switched off.
 - Return proper errors with useful messages. “No data for 2031-01-01” and not a 500.
-

PART 8 — ML PIPELINE

Same as before, now with a clean handoff at the end.

```
src/
├─ data/           fetch and harmonise satellites + GLORYS → zarr
├─ datasets/       patch sampler
├─ models/         encoder + depth decoder
├─ losses/         CRPS, smoothness, density
├─ eval/           Argo matching, metrics, diagnostics
├─ baselines/      climatology, linear, gradient boosting
├─ train.py
└─ export.py       ← runs the model over the test period, writes predictions.zarr
```

`export.py` is the contract with the rest of the team. It should be written in the first week, even when the model is still bad, so the backend and frontend have something real to work with.

Data volume. Today's ocean and tomorrow's ocean are nearly identical, so daily data is mostly redundant. Take every 5th day, 2010–2019. That is around 19 GB to download and about 2 GB on disk once processed. Downloading every day for ten years would be several hundred GB and would teach the model almost nothing extra.

A useful shortcut: GLORYS comes at $1/12^\circ$ resolution. We need 0.25° . Since $1/12 \times 3 = 1/4$ exactly, coarsening by a factor of three is all the regridding we ever need — no special libraries.

PART 9 — WHO DOES WHAT

Six people, three tracks, everyone paired so nothing has a single owner.

	Person	Owns	Hands off
ML	M1	model architecture, losses, training, <code>export.py</code>	<code>predictions.zarr</code> , <code>model.pt</code>
	M2	Argo pipeline, validation, metrics, baselines, physics diagnostics	<code>metrics.json</code>
Data	D1	all satellite + GLORYS fetching, harmonisation, climatology, land mask	<code>surface.zarr</code> , <code>glorys.zarr</code>
Backend	B1	FastAPI, PNG rendering, caching, Docker, the NRT endpoint	OpenAPI schema, <code>fixtures/</code>
Frontend	F1	app shell, map, depth rail, design system, deck	the deployed site
	F2	profile chart, cross-section, cyclone screen, validation screen	

Pairings: M1+M2 · D1+B1 · F1+F2.

Rules:

- M2 does not read the training code and does not tune anything. The validation stays honest by staying separate.
- F1 and F2 build against `fixtures/` from hour one. They never wait for the model.
- B1 is the bridge. If ML and frontend disagree about a data format, B1 decides.
- One branch per person, pull requests into main, no direct pushes.
- Fifteen-minute standup daily. Blockers only.

PART 10 — TIMELINE

All three tracks run at once, from day one. Nobody waits.

	ML + Data	Backend	Frontend
Days 1-2	repo, env, fake data generator	endpoint shapes, fixtures	design tokens, app shell, map renders
Days 3-5	real data downloaded and gridded	serving PNGs from fake zarr	depth rail working, profile chart on fixtures
Days 6-8	model trains, beats linear baseline	wired to real predictions	cyclone screen, cross-section
Days 9-10	validation, three-way table	caching, Docker, offline test	validation screen, polish
Days 11-12	retrain with everything	pre-warm cache	rehearse, record backup video

Grand finale, 36 hours. Arrive with the data on two separate drives, checkpoints committed, the whole stack running offline, and a recorded video on somebody's phone. **Freeze the model at hour 28.** No new training after that, no exceptions. Spend the last hours rehearsing out loud with someone playing a hostile judge.

PART 11 — THE FIVE-MINUTE DEMO

Rehearse this until it is automatic. One person talks, the others stay quiet.

1. **(30s)** Open on the map, 18 May 2020, at the surface. “This is what a satellite sees. Thirty degrees, almost everywhere. It looks the same everywhere.” Point at two spots.
 2. **(45s)** Drag the depth rail down to 100 m. The map transforms. “This is what is actually there. These two places are completely different, and no satellite can tell you that.”
 3. **(60s)** Click a point. Profile appears with the uncertainty band. Argo dots appear on top of our curve. “This is a real float measurement. We never saw it in training.”
 4. **(90s)** Cyclone screen. Amphan’s track animates. Cross-section opens. Beside it, the flat SST line. “Amphan intensified here. Surface temperature does not explain why. The depth of the warm layer does.”
 5. **(45s)** Validation screen. The three-way table. “We are not claiming to beat the reanalysis. We are claiming to match it from today’s satellite data alone, without any of the float data it needs, and here is exactly what that costs.”
 6. **(30s)** Live screen. Press run. Today’s data goes in, a field comes out, timer showing. “That is real, now, on this laptop.”
-

PART 12 — THINGS THAT WILL GO WRONG

- **Forcing temperature to always decrease with depth.** Deletes the Bay of Bengal winter inversion, which is the phenomenon we are selling.
- **Predicting temperature only.** Then the physics check is impossible and the project name is a lie.
- **Shuffling the data randomly.** Gives a beautiful, fictional error number.
- **Computing the climatology using all years.** Leaks the test set into training.
- **Forgetting the land mask.** Predicting 1000 m temperatures in the Sundarbans on a live map is a bad thirty seconds.
- **Training on model salinity and running on satellite salinity.** The most likely cause of something that validates beautifully and fails on stage.
- **Running the neural network during the demo.** Precompute. Always.
- **Relying on venue wifi.** Everything offline, plus the video.
- **Six people on one branch.** Predictable and avoidable.
- **Hiding the baselines.** Judges assume the worst about numbers you did not show.